National University of Computer & Emerging Sciences
Karachi Campus

Proposal

Courtify – Futsal Court Booking Platform

BS(CS) - 5F

Team Lead: Zayan Ahmed 23K-0891
Hammad Uddin 23K-0863
Hashir Khan 23K-0684

Advisor: Asst. Prof. Engr. Abdul Rahman

Date: 16-02-2026

Contents

1. Overview 3
2. Project Detail 3
2.1 Problem or Challenge 3
2.2 Project Vision 3
2.3 Project Feasibility Study 3
2.3.1 Technical Feasibility 3
2.3.2 Operational Feasibility 4
2.3.3 Economic Feasibility 4
2.4 Project Goals 4
2.5 Project Abstract 5
2.6 Project Scope 5
2.7 Project Team 6
2.8 Existing & Related Work 6
2.9 Proposed Solution & Architecture 7
2.10 Use Case Model 9
2.11 Feature List 10
2.12 Software Development Methodology 11
2.13 Proposed Infrastructure, Platform and Tools 11
2.14 Project Timeline 12
2.15 Mockups (Optional) ..... **Error! Bookmark not defined.**
2.16 References 12

1. Overview

Courtify will be a web-based futsal booking system designed to make sports reservations simple, fast, and reliable. The MVP will focus on core features: booking futsal courts, managing payments, enabling cancellations, sending email notifications via SMTP, and supporting basic matchmaking.

The goal is to replace the outdated process of booking through WhatsApp groups, phone calls, and manual records. This system offers an automated, user-friendly, and accessible platform for students and sports players. The project targets students, young players, court owners, admins, and solo players looking for a streamlined digital experience.

The project delivers a new product built from scratch. The tech stack includes PostgreSQL, Flask, React, and SMTP-based notifications. The system is stable, secure, and scalable for future development into a full multi-sport platform.

Estimated Project Duration: 10-weeks for a complete and functional MVP

2. Project Detail

2.1 Problem or Challenge

In Pakistan, especially in cities like Karachi, booking a futsal court is still done through WhatsApp groups, phone calls, and manual record-keeping. This process is unreliable, time-consuming, and often leads to double bookings, miscommunication, and no-shows. Court owners have no efficient way to manage their bookings digitally, and players waste time coordinating schedules manually.

Additionally, solo players who want to join a match have no organized platform to find open games. The absence of a centralized booking system creates friction for all stakeholders – players, court owners, and administrators alike.

2.2 Project Vision

Courtify envisions becoming the go-to digital platform for futsal court booking in Pakistan. By automating the reservation process and providing real-time court availability, payment integration, and matchmaking capabilities, Courtify aims to eliminate the inefficiencies of manual booking. The ultimate goal is to create a seamless, mobile-friendly ecosystem where players can discover, book, and enjoy futsal courts with a single click – and where court owners can manage their operations effortlessly.

2.3 Project Feasibility Study

A feasibility study assesses the operational, technical, and economic merits of the proposed project to determine if it is worthy of proceeding to the development phase.

2.3.1 Technical Feasibility

The project uses well-established, open-source technologies: PostgreSQL for the database, Flask (Python) for the backend API, and React (JavaScript) for the frontend. SMTP is used for email notifications and Stripe for payment processing. All chosen technologies have extensive

documentation, community support, and are free to use. The development team has working knowledge of these technologies from coursework. The system architecture follows a clean, modular design that is straightforward to develop and maintain.

2.3.2 Operational Feasibility

The target users (students, young players, court owners) are already familiar with web and mobile applications. The system provides an intuitive interface where users can browse courts, make bookings, and receive confirmations via email. Court owners benefit from a digital management dashboard. The system is designed to be mobile-friendly, which aligns with user behavior in the target demographic. Minimal training is required for adoption.

2.3.3 Economic Feasibility

The project leverages entirely free and open-source tools (PostgreSQL, Flask, React), significantly reducing development costs. Hosting can be done on affordable cloud platforms. The Stripe payment gateway charges standard transaction fees. The total infrastructure cost is minimal, making this project economically viable as a student project with potential for commercialization.

2.4 Project Goals

Functional Goals:

- Provide a complete court booking lifecycle: browse, book, pay, cancel, and receive notifications.
- Enable court owners to manage their courts and view booking details digitally.
- Support matchmaking for solo players to join open matches.
- Allow users to mark favorite courts for quick access.

Technological Goals:

- Build a full-stack web application using PostgreSQL, Flask, and React.
- Integrate Stripe for secure payment processing.
- Implement SMTP-based email notifications with retry logic.

Quality Goals:

- Booking confirmation should take less than 2 seconds.
- Database queries should be optimized for multiple concurrent users.
- Booking data must not be lost or duplicated.

Organizational Goals:

- Gain hands-on experience in full-stack development and database design.
- Apply SDLC methodology in a real-world project scenario.

Usability & Portability:

- Interface should be mobile-friendly and straightforward.
- Users should find their bookings in one click.

Constraints:

- Full payment gateway integration (JazzCash, Easypaisa) is not included in the current scope.
- Multi-sport support and comprehensive analytics are deferred to future versions.
- Passwords must be stored as hashes; only authenticated users may access booking features.

2.5 Project Abstract

Courtify is a web-based futsal court booking platform designed to digitize and streamline the sports reservation process in Pakistan. The current system of booking courts through WhatsApp groups, phone calls, and manual record-keeping is unreliable and leads to double bookings, miscommunication, and no-shows. Courtify addresses these problems by providing an automated, user-friendly, and accessible platform. The system enables users to browse available courts, make bookings with Stripe-based payments, cancel reservations within a defined time window, and receive real-time email notifications via SMTP. Court owners can manage their venues and view booking details through a dedicated dashboard. A matchmaking feature allows solo players to join open games. The admin module provides oversight of all platform activity and user management. The technology stack comprises PostgreSQL as the relational database (normalized to 3NF), Flask (Python) as the backend framework, and React (JavaScript) for the frontend. The system architecture follows a modular, scalable design that supports future expansion to multi-sport venues. The database schema cleanly separates users, courts, bookings, payments, notifications, matchmaking, and favourites entities, with foreign keys enforcing referential integrity. Non-functional requirements address performance (sub-2-second booking confirmations), reliability (email retry logic), security (password hashing, authentication), and scalability. Courtify replaces outdated manual workflows with a modern digital solution, improving the experience for players, court owners, and administrators alike.

2.6 Project Scope

The scope of the project includes:

User Features:

- Register / Login
- View available courts
- Book a court
- Stripe payment
- Cancel booking with rules (1-hour window)
- Receive booking notifications via SMTP
- Join matchmaking queue
- Add favourite courts

Owner Features:

- Manage their courts
- View court bookings
- Receive notifications

Admin Features:
- Manage users
- View platform activity

System Features:
- Auto-send reminders
- Retry failed email notifications

The scope does not include:
- Full payment gateway integration (JazzCash, Easypaisa)
- Multi-sport support
- Comprehensive analytics

These may be added in future versions.

2.7 Project Team

Organization: Name	Availability	Comment
FAST: Zayan Ahmed	Part-time	Backend, DB
FAST: Hammad Uddin	Part-time	Frontend
FAST: Hashir Khan	Part-time	Backend, DB

2.8 Existing & Related Work

Below is a comparison of existing solutions and how Courtify differentiates itself:

Feature	WhatsApp Booking	Phone Call Booking	Generic Booking Apps	Courtify
Online Court Browsing	No	No	Yes	Yes
Stripe Payment	No	No	Varies	Yes
Cancellation Rules	No	No	Varies	Yes (1-hr window)
Email Notifications	No	No	Varies	Yes (SMTP)

Matchmaking	No	No	No	Yes
Favourites	No	No	Varies	Yes
Owner Dashboard	No	No	Varies	Yes
Focused on Futsal (PK)	No	No	No	Yes

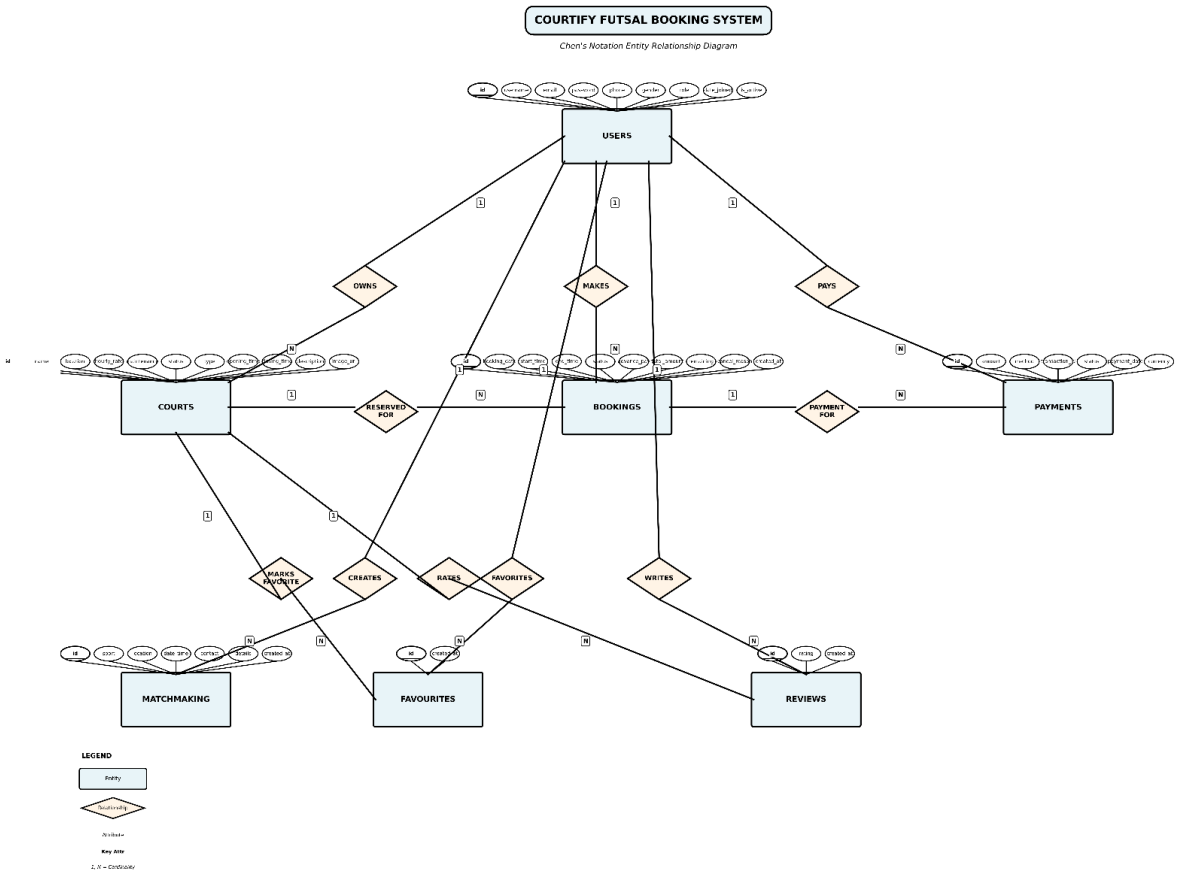
Courtify's uniqueness lies in its targeted focus on the futsal ecosystem in Pakistan, combining real-time booking, payment, matchmaking, and notification features in one platform.

2.9 Proposed Solution & Architecture

Courtify follows a three-tier architecture:

- Frontend (Presentation Layer): React.js provides a responsive, mobile-friendly user interface for browsing courts, making bookings, and managing accounts.
- Backend (Application Layer): Flask (Python) serves as the REST API layer, handling business logic including booking management, payment processing via Stripe, matchmaking, and SMTP-based notifications with retry.
- Database (Data Layer): PostgreSQL stores all data in a normalized (3NF) schema covering users, courts, bookings, payments, notifications, matchmaking, and favourites.

ER Diagram:


COURTIFY FUTSAL BOOKING SYSTEM
Chen's Notation Entity Relationship Diagram
USERS
OWNS
MAKES
PAYS
COURTS
RESERVED FOR
BOOKINGS
PAYMENT FOR
PAYMENTS
MARKS FAVOURITE
CREATES
RATES
FAVORITES
WRITES
MATCHMAKING
FAVOURITES
REVIEWS
LEGEND
Entity
Relationship
Attribute
Key Attr
1, N – Cardinality

The schema cleanly separates users, courts, bookings, payments, notifications, matchmaking, and favourites. Foreign keys handle all relationships without data duplication, and the schema prevents inconsistencies while being easy to extend.

2.10 Use Case Model

Actors: User (Player), Court Owner, Admin, System (Background)

User (Player) Use Cases:

- Create an account and log in
- Browse all available courts
- Book a court and make an advance payment
- Cancel a booking within allowed time
- Join an open match (matchmaking)
- Add courts to favourites
- Receive email notifications for bookings and updates

Court Owner Use Cases:

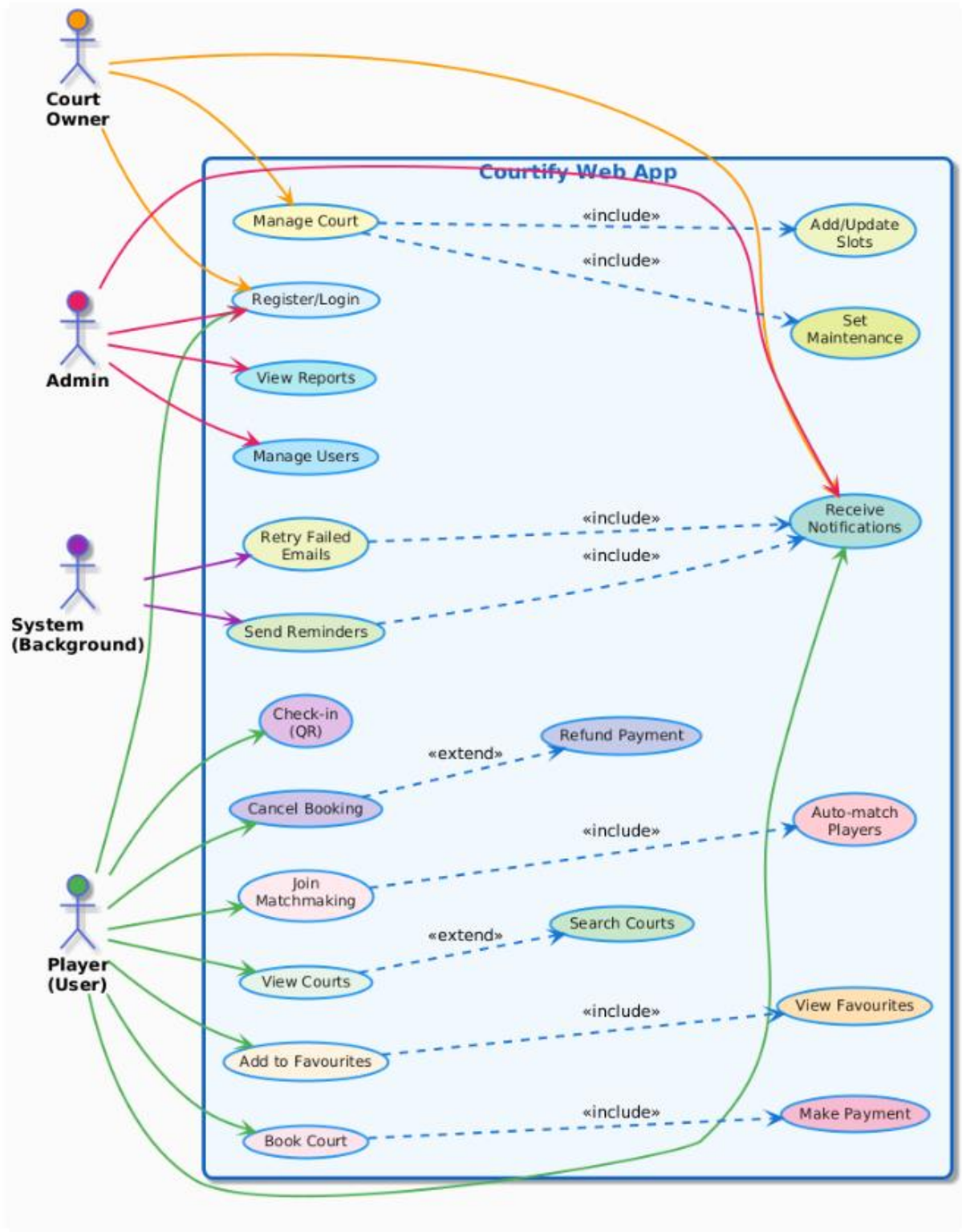
- Register as an owner
- Add or update court information
- View booking details for their courts

Admin Use Cases:

- Manage all users
- View system activity and reports

System (Background) Use Cases:

- Send automatic reminder emails
- Retry failed SMTP notifications



2.11 Feature List

Below are the 20 core domain-related features of Courtify:

1. User Registration with role selection (Player / Owner)
2. Court Browsing with filters (location, availability, price)
3. Real-time Court Availability Check
4. Court Booking with date and time slot selection
5. Stripe-based Advance Payment Processing
6. Booking Cancellation with 1-hour window enforcement
7. Automatic Refund Processing on valid cancellation
8. Email Booking Confirmation via SMTP
9. Email Reminder Notifications before match time
10. Failed Email Notification Retry Mechanism
11. Matchmaking Queue for solo players
12. Auto-match solo players into open games
13. Add Courts to Favourites
14. Court Owner: Add New Court with details (location, price, type)
15. Court Owner: Update Court Information
16. Court Owner: View Booking Calendar for their courts
17. Court Owner: Receive Booking Notification
18. Admin: User Management (view, activate, deactivate)
19. Admin: Platform Activity Monitoring and Reports
20. Court Reviews and Ratings by players

2.12 Software Development Methodology

The project follows an Agile (Iterative) development methodology, specifically adapted for a small

academic team. Development is organized into sprints, each focused on delivering a functional

increment of the system:

• Sprint 1: Database schema design, normalization (3NF), and PostgreSQL setup.

• Sprint 2: Backend API development (Flask) – authentication, court CRUD, booking logic.

• Sprint 3: Frontend development (React) – user interface for browsing, booking, and

dashboards.

• Sprint 4: Payment integration (Stripe), email notifications (SMTP), and matchmaking.

• Sprint 5: Testing, bug fixes, and final integration.

This iterative approach allows the team to continuously test and refine features, incorporate

feedback, and deliver a working product incrementally.

2.13 Proposed Infrastructure, Platform and Tools

Category	Tools / Technologies
Frontend	React.js, HTML5, CSS3, JavaScript
Backend	Flask (Python)
Database	PostgreSQL
Payment Gateway	Stripe
Email / Notifications	SMTP (Gmail SMTP)
Version Control	Git, GitHub
IDE / Editor	VS Code
Modeling / Diagrams	draw.io, Papyrus, HTML
API Testing	Postman
Deployment	Frontend-Vercel, Backend-Vercel
Project Management	WhatsApp, Github

2.14 Project Timeline

Sprint	Tasks	Duration
Sprint 1	Database schema design, normalization (3NF), PostgreSQL setup	Week 1–2
Sprint 2	Backend API: authentication, court CRUD, booking logic	Week 3–4
Sprint 3	Frontend: React UI for browsing, booking, dashboards	Week 5–6
Sprint 4	Stripe payment, SMTP notifications, matchmaking	Week 7–8
Sprint 5	Testing, bug fixes, final integration, documentation	Week 9–10

2.16 References

[1] PostgreSQL Global Development Group, “PostgreSQL Documentation,” [Online]. Available: https://www.postgresql.org/docs/

[2] Pallets Projects, “Flask Official Documentation,” [Online]. Available: https://flask.palletsprojects.com/

[3] Meta Platforms, “ReactJS Documentation,” [Online]. Available: https://react.dev/

[4] J. Klensin, “Simple Mail Transfer Protocol,” RFC 5321, Internet Engineering Task Force, 2008.

[5] Class lecture notes and course guidelines from FAST-NUCES.